

Técnicas digitales I

Micro IO - 2024

Condiciones generales

1. Los nombres de todos los archivos VHDL tendrán el mismo nombre que su entidad
2. El clock es activo flanco ascendente
3. Las señales/ports resets, enable, carga paralela, etc son activas nivel alto.
4. El dispositivo usado es xc7z010iclg400-1L

Características

1. Dato: 16 bits
2. Memoria de datos: 64K word (máxima)
3. Memoria de código: 64K word (máxima)
4. Port de I/O:
 - a. 1 ports de salida 16 bits
 - b. 1 ports de entrada 16 bits
5. Uart 9600 8N1
6. ALU: Puede realizar las siguientes operaciones
 - a. Operaciones aritméticas: Suma y resta con operando de 16 Bits con Carry.
 - i. Maneja los Bits: Carry (C), Overflow (O); Cero (Z); Negative (N)
 - ii. Tiene la posibilidad de aritmética con saturación (Bit sat)
 - b. Operaciones Lógicas and, or y xor

Diagrama en bloques

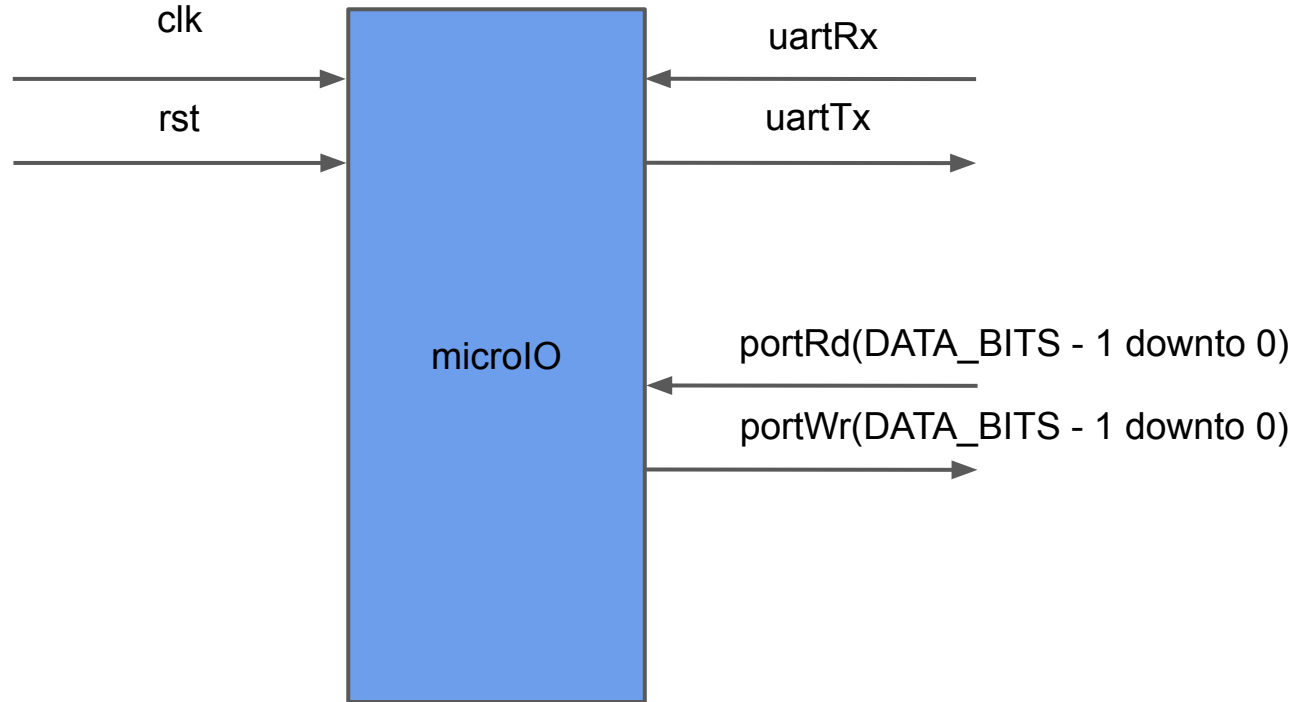
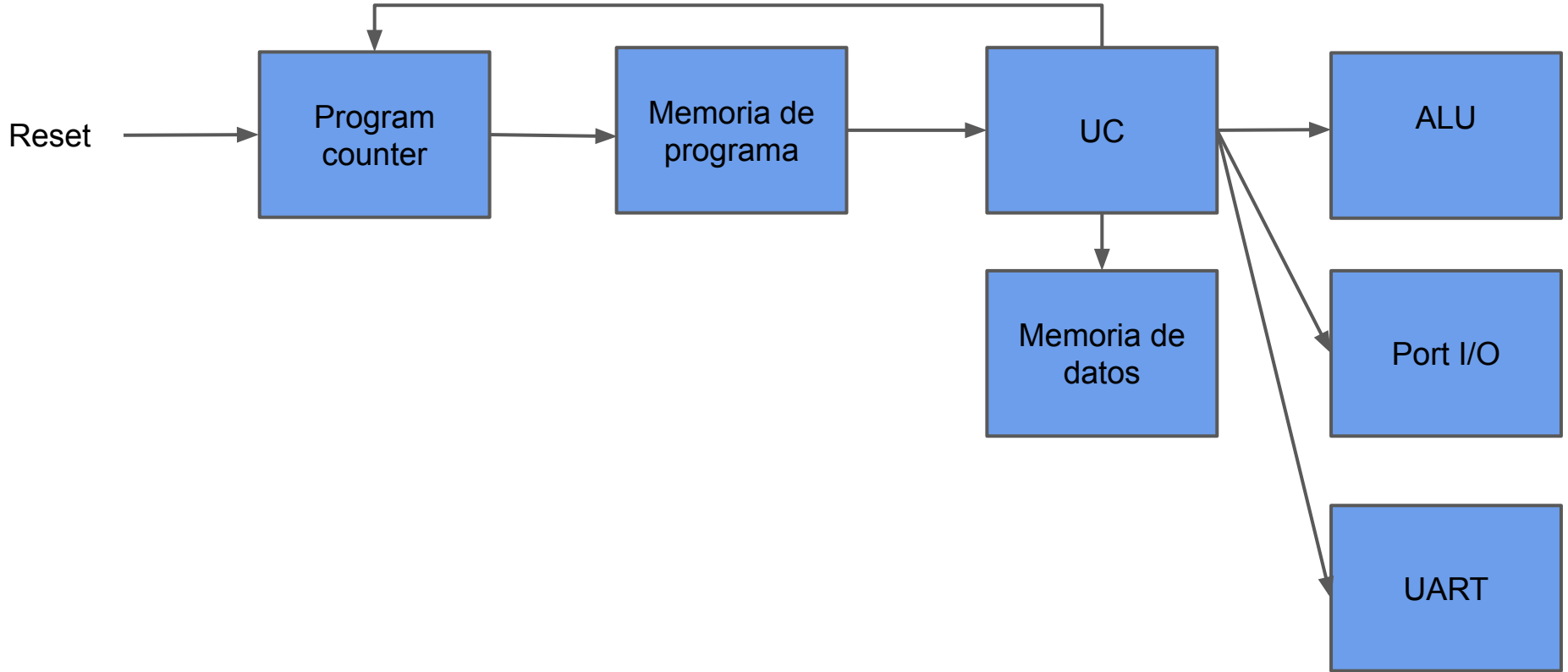


Diagrama en bloques



Notación

Simbolo	Descripción
MEM	Dirección de una posición de memoria RAM o de programa
LIT	Constante de 16 bits
PORT	Port de salida y entrada
UART	Registro de la UART
ACC	Acumulador. (Registro de salida de la ALU)
FLAG	Puede ser Z(Cero); OF (Overflow); C (Carry); N (Negative)
*MEM	Contenido de la posición de memoria
SAT	Saturación del resultado de la operación

Set de instrucciones ALU - lógica

Nemónico	OPA	OPB	Descripción	Flags
and	MEM	ACC	ACC = ACC and *MEM	Z;N
and	LIT	ACC	ACC = ACC and LIT	Z;N
or	MEM	ACC	ACC = ACC and *MEM	Z;N
or	LIT	ACC	ACC = ACC and LIT	Z;N
xor	MEM	ACC	ACC = ACC and *MEM	Z;N
xor	LIT	ACC	ACC = ACC and LIT	Z;N

Set de instrucciones ALU - Aritméticas

Nemónico	OPA	OPB	Descripción	Flags
addc	MEM	ACC	$(\text{Carry}, \text{ACC}) = \text{ACC} + * \text{MEM} + \text{Carry}, \text{ si sat} = 0$ $(\text{Carry}, \text{ACC}) = \text{SAT} (\text{ACC} + * \text{MEM} + \text{Carry}), \text{ si sat} = 1$	C;N;Z;OF
addc	LIT	ACC	$(\text{Carry}, \text{ACC}) = \text{ACC} + * \text{LIT} + \text{Carry}, \text{ si sat} = 0$ $(\text{Carry}, \text{ACC}) = \text{SAT} (\text{ACC} + * \text{LIT} + \text{Carry}), \text{ si sat} = 1$	C;N;Z;OF
subc	MEM	ACC	$(\text{Borrow}, \text{ACC}) = \text{ACC} - * \text{MEM} + \text{Borrow}, \text{ si sat} = 0$ $(\text{Borrow}, \text{ACC}) = \text{SAT} (\text{ACC} - * \text{MEM} + \text{Borrow}), \text{ si sat} = 1$	C;N;Z;OF
subc	LIT	ACC	$(\text{Borrow}, \text{ACC}) = \text{ACC} - \text{LIT} + \text{Borrow}, \text{ si sat} = 0$ $(\text{Borrow}, \text{ACC}) = \text{SAT} (\text{ACC} - \text{LIT} + \text{Borrow}), \text{ si sat} = 1$	C;N;Z;OF
rotc	MEM	ACC	$(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \ll * \text{MEM}, \text{ Si } * \text{MEM} \geq 0$ $(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \gg * \text{MEM}, \text{ Si } * \text{MEM} < 0$	C;N;Z
rotc	LIT	ACC	$(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \ll \text{LIT}, \text{ Si } \text{LIT} \geq 0$ $(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \gg \text{LIT}, \text{ Si } \text{LIT} < 0$	C;N;Z
accLoad	MEM	ACC	$\text{ACC} = * \text{MEM}$	Z;N
accLoad	MEM	LIT	$\text{ACC} = \text{LIT}$	Z;N

Set de instrucciones ALU - flags

Nemónico	OPA	OPB	Descripción	Flags
carrySet	LIT		$C = \text{LIT} (0)$	C
satSet	LIT		$\text{sat} = \text{LIT} (0)$	-

Set de instrucciones jmp

Nemónico	OPA	OPB	Descripción	Flags
jmpCondZ	MEM	ZERO	if (Z == 1) PC = *MEM	
jmpCondZ	LIT	ZERO	if (Z == 1) PC = LIT	
jmpCondOV	MEM	ZERO	if (OV == 1) PC = *MEM	
jmpCondOv	LIT	OVERFLOW	if (OV == 1) PC = LIT	
jmpCondCB	MEM	CARRY_BORROW	if (C == 1) PC = *MEM	
jmpCondCB	LIT	jmpCondCB	if (C == 1) PC = LIT	
jmpCondN	MEM	NEGATIVE	if (N == 1) PC = *MEM	
jmpCondN	LIT	NEGATIVE	if (N == 1) PC = LIT	
jmp	MEM		PC = *MEM	
jmp	LIT		PC = LIT	

Set de instrucciones mov

Nemónico	OPA	OPB	Descripción	Flags
read	MEM	PORT	*MEM = PORT	
readUart	MEM	UART	*MEM = UART	
write	MEM	PORT	PORT = *MEM	
write	LIT	PORT	PORT = LIT	
writeUart	MEM	UART	UART = *MEM	
writeUart	LIT	UART	UART = LIT	
mov	MEM	ACC	*MEM = ACC	
mov	MEM	0	*MEM = 0	
nop			No hace nada	

Program counter

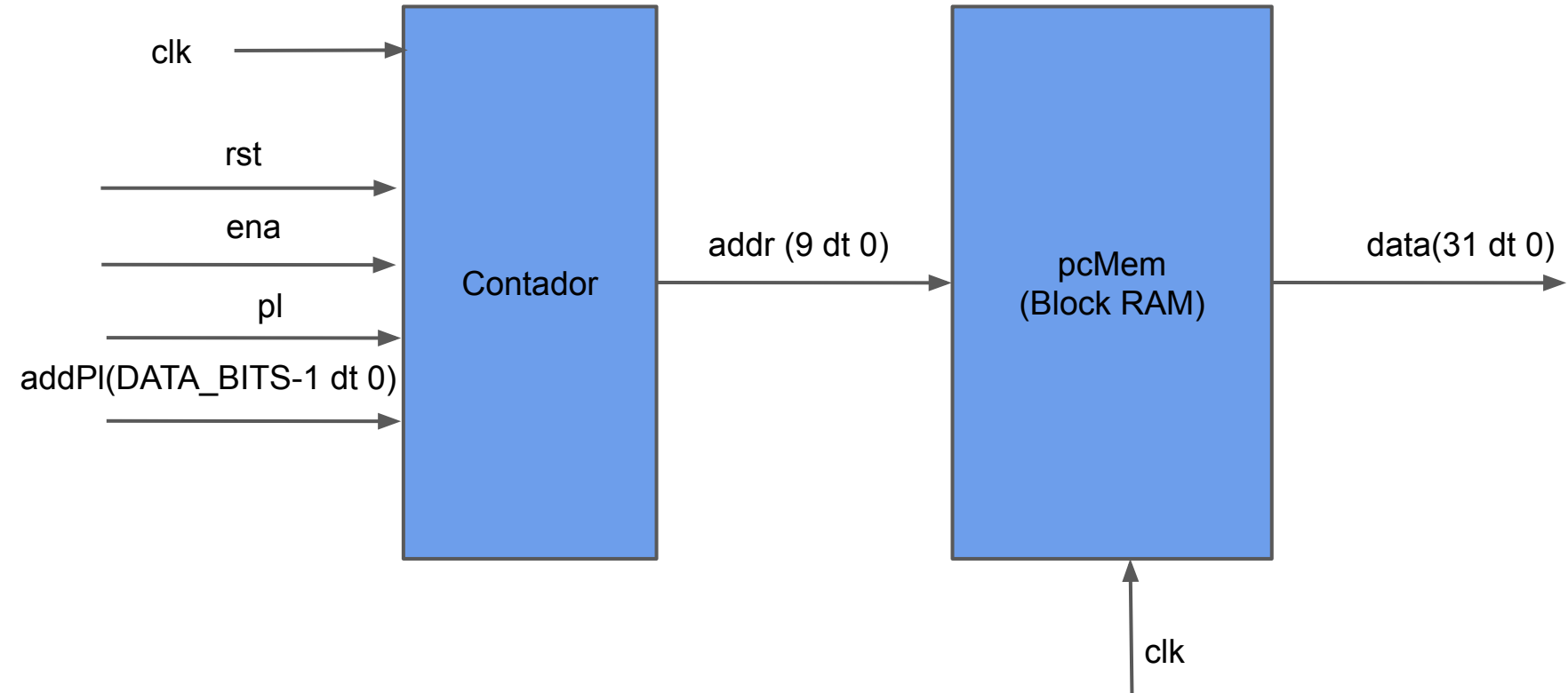
Características

1. Este módulo direccionara una block ram solo lectura
2. La block ram tiene 32 bits de datos y hasta 1024 posiciones.
3. Inicia el direccionamiento de la memoria en la posición cero.
4. Al superar la máxima posición de memoria se vuelve a la posición cero.
5. Permite la carga paralela de una dirección de la block ram

Entidad

```
entity pc is
  Generic ( DATA_BITS : integer := 16);
  Port ( clk : in std_logic; -- Clock
        rst : in std_logic; -- Reset sincronico
        ena : in std_logic; -- Enable, cuando está habilitado incrementa el addr en 1
        pl : in std_logic; -- Habilita la carga paralela del addrPI
        plAddr : in std_logic_vector (DATA_BITS - 1 downto 0); -- Direccion a cargar
        data: out std_logic_vector (31 downto 0) -- Dato leído de la memoria
  );
end pc;
```

Diagrama en bloques



Port

Características

1. Un port de entrada.
2. Un port de salida.
3. Los port de salida y entrada están registrados.

Entidad

entity portIO is

```
    Port ( clk : in std_logic;  
          rst : in std_logic;  
          portRd : in std_logic_vector (7 downto 0);  
          portRdReg : out std_logic_vector (7 downto 0);  
          portWrEna : in std_logic;  
          portWr : out std_logic_vector (7 downto 0);  
          portWrReg : in std_logic_vector (7 downto 0));
```

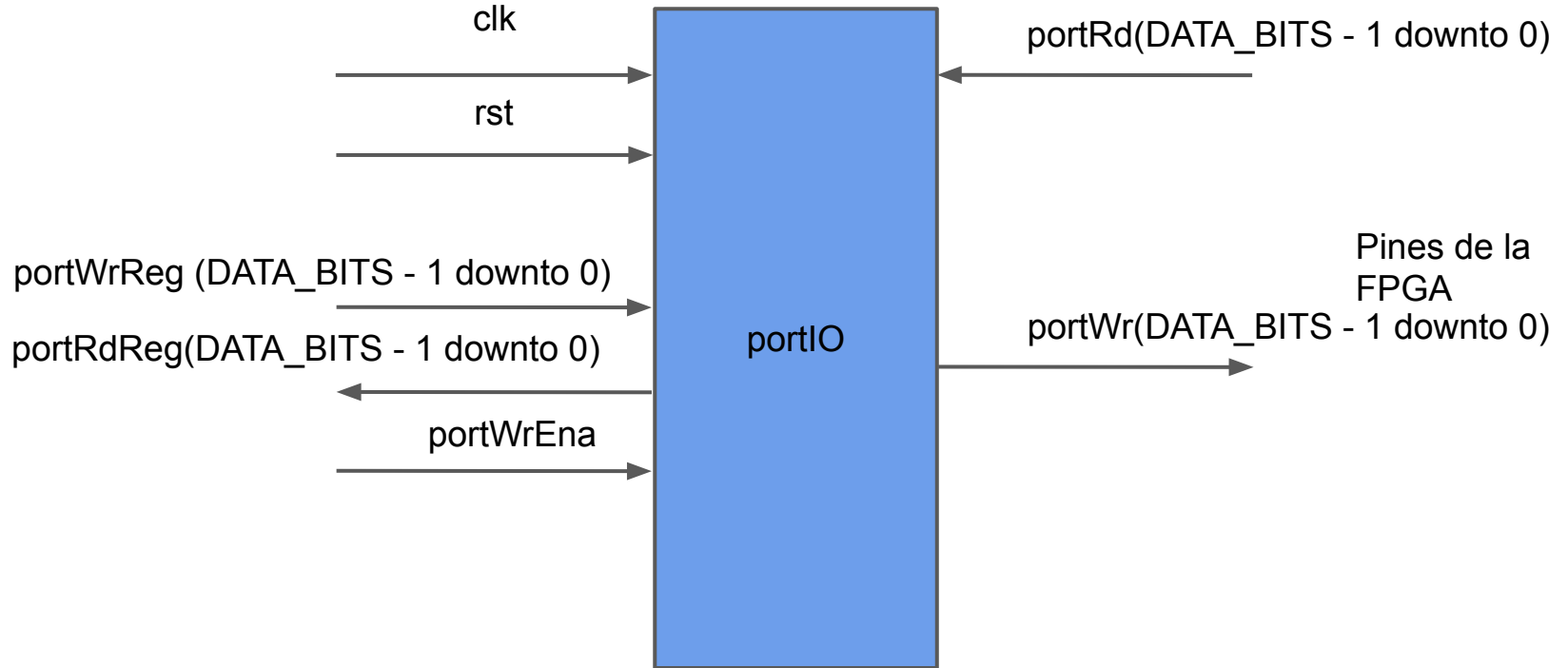
end portIO;

Entidad

Donde:

- portRd: Port de entrada del procesador
- portWr: Port de salida del procesador
- portRdReg: Entrada para escribir en el port de salida.
- portWrReg: Salida para leer el port
- portWrEna: Habilita la escritura en el port

Diagrama en bloques



ALU

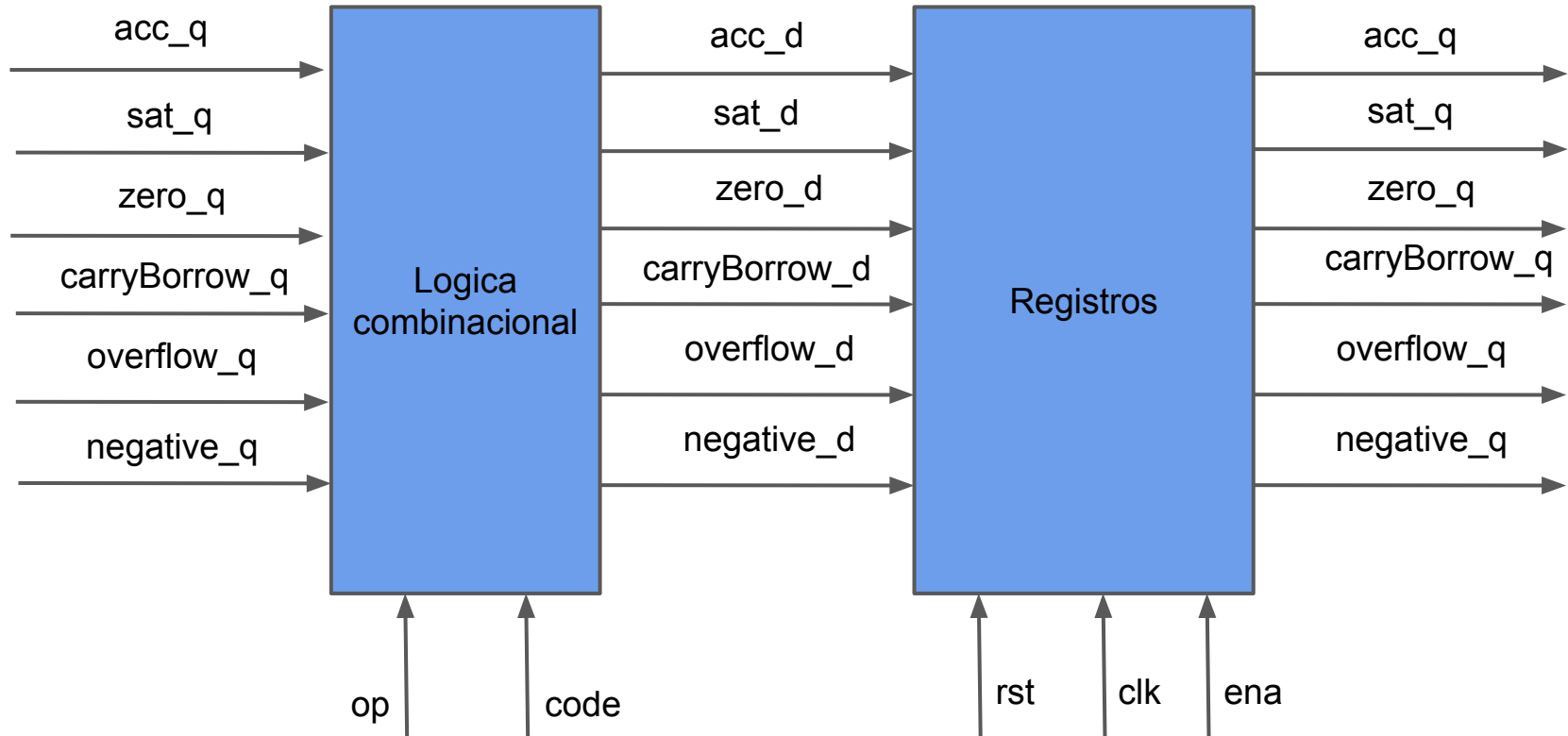
Características

1. Acumulador: 16 bits
2. Operando: 16 bits
3. Operaciones aritméticas:
 - a. Suma con carry
 - b. Resta con borrow
4. Operaciones Lógicas: and, or, xor
5. Operación de rotación rango 16 a -16
6. Flags: carryBorrow, overflow, zero, negative
7. Carga del acumulador.
8. Set y clr del carry
9. Saturación de las operaciones de suma y resta

Entidad

```
entity alu
generic (DATA_BITS: integer := 16);
port ( clk : in std_logic;                -- Clock del sistema
      rst : in std_logic;                -- Reset sincrónico del sistema
      ena : in std_logic;                -- Habilita la realización de la operación
      code: in std_logic_vector (3 downto 0); -- Código de la operación a realizar
      op  : in std_logic_vector (DATA_BITS-1 downto 0); -- Operando
      acc : out std_logic_vector (DATA_BITS-1 downto 0); -- Acumulador
      zero : out std_logic;                -- Acumulador en cero
      overflow : out std_logic;            -- Overflow
      carryBorrow : out std_logic;        -- Carry/Borrow
      negative : out std_logic;            -- Acumulador negativo
    );
end alu;
```

Diagrama en bloques



Operaciones

Code	Operación	Descripción
0	$ACC = ACC \text{ and } op$	and bit a bit
1	$ACC = ACC \text{ or } op$	or bit a bit
2	$ACC = ACC \text{ xor } op$	xor bit a bit
3	$(carry, ACC) = ACC + op + carry$	Suma con carry in y carry out Si el bit sat está en uno se satura el resultado de la operación
4	$(borrow, ACC) = ACC - op + borrow$	Resta con borrow in y borrow out Si el bit sat está en uno se satura el resultado de la operación
5	$(carry, ACC) \text{ rot } op$	Rotación con carry (op vale entre -16 y 16)
6	$ACC = op$	Carga ACC con op
7	$carrySet = op$	Setea el carry en 1 ó 0
8	$satSet = op$	Configura la saturación del acumulador al sumar o restar
10 a 15	Uso futuro	Uso futuro

Flags

Cod e	Operación	carryBorrow	zero	negative	overflow
0	ACC = ACC and op	Sin cambio	1, si ACC es cero	1, si ACC es negativo	Sin cambio
1	ACC = ACC or op	Sin cambio	1, si ACC es cero	1, si ACC es negativo	Sin cambio
2	ACC = ACC xor op	Sin cambio	1, si ACC es cero	1, si ACC es negativo	Sin cambio
3	(carry,ACC) = ACC + op + carry	Carry	1, si ACC es cero	1, si ACC es negativo	Overflow de la operación
4	(borrow,ACC) = ACC + op + borrow	borrow	1, si ACC es cero	1, si ACC es negativo	Overflow de la operación

Flags

Code	Operación	carryBorrow	zero	negative	overflow
5	$(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \ll \text{op}$, Si $\text{op} \geq 0$ $(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \gg \text{op}$, Si $\text{op} < 0$	Si $\text{op} = 0$, Carry sin cambio Si $\text{op} > 0$, $\text{Carry} = \text{ACC} (16 - \text{op})$ Si $\text{op} < 0$, $\text{Carry} = \text{ACC} (-1 - \text{op})$	1, si ACC es cero	1, si ACC es negativo	Sin cambio
6	$\text{ACC} = \text{op}$	Sin cambio	1, si ACC es cero	1, si ACC es negativo	Sin cambio
7	carrySet	$\text{op}(0)$	Sin cambio	Sin cambio	Sin cambio
8	satSet	Sin cambio	Sin cambio	Sin cambio	Sin cambio
9 a 15	Uso futuro	Sin cambio	Sin cambio	Sin cambio	Sin cambio

Unidad de control

Características

1. Leer el programa
2. Decodificar
3. Ejecutar la instrucción

Entidad

```
entity uc is
generic (DATA_BITS: integer := 16;
        UART_DATA_BITS: integer := 8);
Port ( clk : in std_logic;
      rst : in std_logic;
      portRd : in std_logic_vector (DATA_BITS - 1 downto 0);
      portWr : out std_logic_vector (DATA_BITS - 1 downto 0);
      portWrEna : out std_logic;
      portAddr : out std_logic_vector (0 downto 0);
      uartDataWr : in std_logic;
      uartDataTx : in std_logic_vector (UART_DATA_BITS - 1 downto 0);
      uartDataRx : out std_logic_vector (UART_DATA_BITS - 1 downto 0);
      aluEna : out std_logic;
      aluCode: out std_logic_vector (3 downto 0);
      aluOp : out std_logic_vector (DATA_BITS - 1 downto 0);
      aluAcc : in std_logic_vector (DATA_BITS - 1 downto 0);
      aluZero : in std_logic;
      aluOverflow : in std_logic;
      aluCarryBorrow : in std_logic;
      aluNegative : in std_logic;
      pcEna : out std_logic;
      pcPI : out std_logic;
      pcPIAddr : out std_logic_vector (15 downto 0);
      pcData: in std_logic_vector (31 downto 0));
end uc;
```

Tipos de instrucciones

- 0 - No operación (NOP)
- 1 - Lógica y aritmética (ALU)
- 2 - Saltos (JMP)
- 3 - Lectura/escritura de RAM y ports. (MOV)

Tipos de instrucciones

0 - No operación (NOP)

1 - Lógica y aritmética (ALU)

2 - Saltos (JMP)

3 - Lectura/escritura de RAM y ports. (MOV)

pcData (31 downto 23)	pcData (22 downto 21) Tipo de operación	pcData (20 downto 17) Código de operación	pcData (16) Selección entre Memoria o literal	pcData (15 downto 0) Memoria o literal
Siempre cero Uso futuro	00. NOP 01.ALU 10.JMP 11.MOV	opCode	0: MEM 1:LIT	MEM/LIT

Instrucción NOP

pcData (31 downto 23)	pcData (22 downto 21) Tipo de operación	pcData (20 downto 17) Código de operación	pcData (16) Selección entre Memoria o literal	pcData (15 downto 0) Memoria o literal
Siempre cero Uso futuro	00. NOP 01.ALU 10.JMP 11.MOV	opCode	0: MEM 1:LIT	MEM/LIT
0	00	0	0	0

Instrucción ALU

pcData (31 downto 23)	pcData (22 downto 21) Tipo de operación	pcData (20 downto 17) Código de operación	pcData (16) Selección entre Memoria o literal	pcData (15 downto 0) Memoria o literal
Siempre cero Uso futuro	00. NOP 01.ALU 10.JMP 11.MOV	opCode	0: MEM 1:LIT	MEM/LIT
0	01	0 - 9	0	MEM
0	01	0 - 9	1	LIT

Instrucción ALU - Operaciones lógicas

pcData (20 dt 17) Código de operación	pcData(16) MEM/LIT	pcData (15 dt 0)	Nemónico	opA	opB	Operación
0	0	MEM	and	MEM	ACC	ACC = ACC and *MEM
0	1	LIT	and	LIT	ACC	ACC = ACC and LIT
1	0	MEM	or	MEM	ACC	ACC = ACC or *MEM
1	1	LIT	or	LIT	ACC	ACC = ACC or LIT
2	0	MEM	xor	MEM	ACC	ACC = ACC xor *MEM
2	1	LIT	xor	LIT	ACC	ACC = ACC xor LIT

Instrucción ALU - Operaciones aritmética

pcData (20 dt 17) Código de operación	pcData(16) MEM/LIT	pcData (15 dt 0)	Nemónico	opA	opB	Operación
3	0	MEM	addc	MEM	ACC	$(\text{Carry}, \text{ACC}) = \text{ACC} + * \text{MEM} + \text{Carry}$
3	1	LIT	addc	LIT	ACC	$(\text{Carry}, \text{ACC}) = \text{ACC} + \text{LIT} + \text{Carry}$
4	0	MEM	subc	MEM	ACC	$(\text{Borrow}, \text{ACC}) = \text{ACC} - * \text{MEM} + \text{Borrow}$
4	1	LIT	subc	LIT	ACC	$(\text{Borrow}, \text{ACC}) = \text{ACC} - \text{LIT} + \text{Borrow}$
5	0	MEM	rotc	MEM	ACC	$(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \ll * \text{MEM}, \text{ Si } * \text{MEM} \geq 0$ $(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \gg * \text{MEM}, \text{ Si } * \text{MEM} < 0$
5	1	LIT	rotc	LIT	ACC	$(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \ll \text{LIT}, \text{ Si } \text{LIT} \geq 0$ $(\text{Carry}, \text{ACC}) = (\text{Carry}, \text{ACC}) \gg \text{LIT}, \text{ Si } \text{LIT} < 0$

Instrucción ALU - Operaciones aritmética

pcData (20 dt 17) Código de operación	pcData(16) MEM/LIT	pcData (15 dt 0)	Nemónico	opA	opB	Operación
6	0	MEM	accLoad	MEM	ACC	ACC = *MEM
6	1	LIT	accLoad	LIT	ACC	ACC = LIT
7	1	LIT	carrySet	LIT	-	carry = LIT(0)
8	1	LIT	satSet	LIT	-	sat = LIT(0)

Instrucción JMP

pcData (31 downto 23)	pcData (22 downto 21) Tipo de operación	pcData (20 downto 17) Código de operación	pcData (16) Selección entre Memoria o literal	pcData (15 downto 0) Memoria o literal
Siempre cero Uso futuro	00. NOP 01.ALU 10.JMP 11.MOV	opCode	0: MEM 1:LIT	MEM/LIT
0	10	0 - 4	0	MEM
0	10	0 - 4	1	LIT

Instrucción JMP

pcData (20 dt 17) Código de operación	pcData(16) MEM/LIT	pcData (15 dt 0)	Nemónico	opA	opB	Operación
0	0	MEM	jmpCondZ	MEM	Zero	IF (ZERO = 1) PC = *MEM
0	1	LIT	jmpCondZ	LIT	Zero	IF (ZERO = 1) PC = LIT
1	0	MEM	jmpCondOv	MEM	Overflow	IF (OVERFLOW = 1) PC = *MEM
1	1	LIT	jmpCondOv	LIT	Overflow	IF (OVERFLOW = 1) PC = LIT
2	0	MEM	jmpCondCB	MEM	Carry	IF (CARRY_BORROW = 1) PC = *MEM
2	1	LIT	jmpCondCB	LIT	Carry	IF (CARRY_BORROW = 1) PC = LIT
3	0	MEM	jmpCondN	MEM	Negative	IF (NEGATIVE = 1) PC = *MEM
3	1	LIT	jmpCondN	LIT	Negative	IF (NEGATIVE = 1) PC = LIT
4	0	MEM	jmp	MEM	-	PC = *MEM
4	1	LIT	jmp	LIT	-	PC = LIT

Instrucción MOV

pcData (31 downto 23)	pcData (22 downto 21) Tipo de operación	pcData (20 downto 17) Código de operación	pcData (16) Selección entre Memoria o literal	pcData (15 downto 0) Memoria o literal
Siempre cero Uso futuro	00. NOP 01.ALU 10.JMP 11.MOV	opCode	0: MEM 1:LIT	MEM/LIT
0	11	0 - 3	0	MEM
0	11	4 - 5	0	MEM
0	11	4 - 5	1	LIT

Instrucción MOV

pcData (20 dt 17) Código de operación	pcData(16) MEM/LIT	pcData (15 dt 0)	Nemónico	opA	opB	Operación
0	0	MEM	read	MEM	PORT	*MEM = PORT
1	0	MEM	readUart	MEM	UART	*MEM = UART
2	0	MEM	mov	MEM	ACC	*MEM = ACC
3	1	LIT	movZ	MEM	-	*MEM = 0
4	0	MEM	write	MEM	PORT	PORT = *MEM
4	1	LIT	write	LIT	PORT	PORT = LIT
5	0	MEM	writeUART	MEM	UART	UART = *MEM
5	1	LIT	writeUART	LIT	UART	UART = LIT

Diagrama en bloques

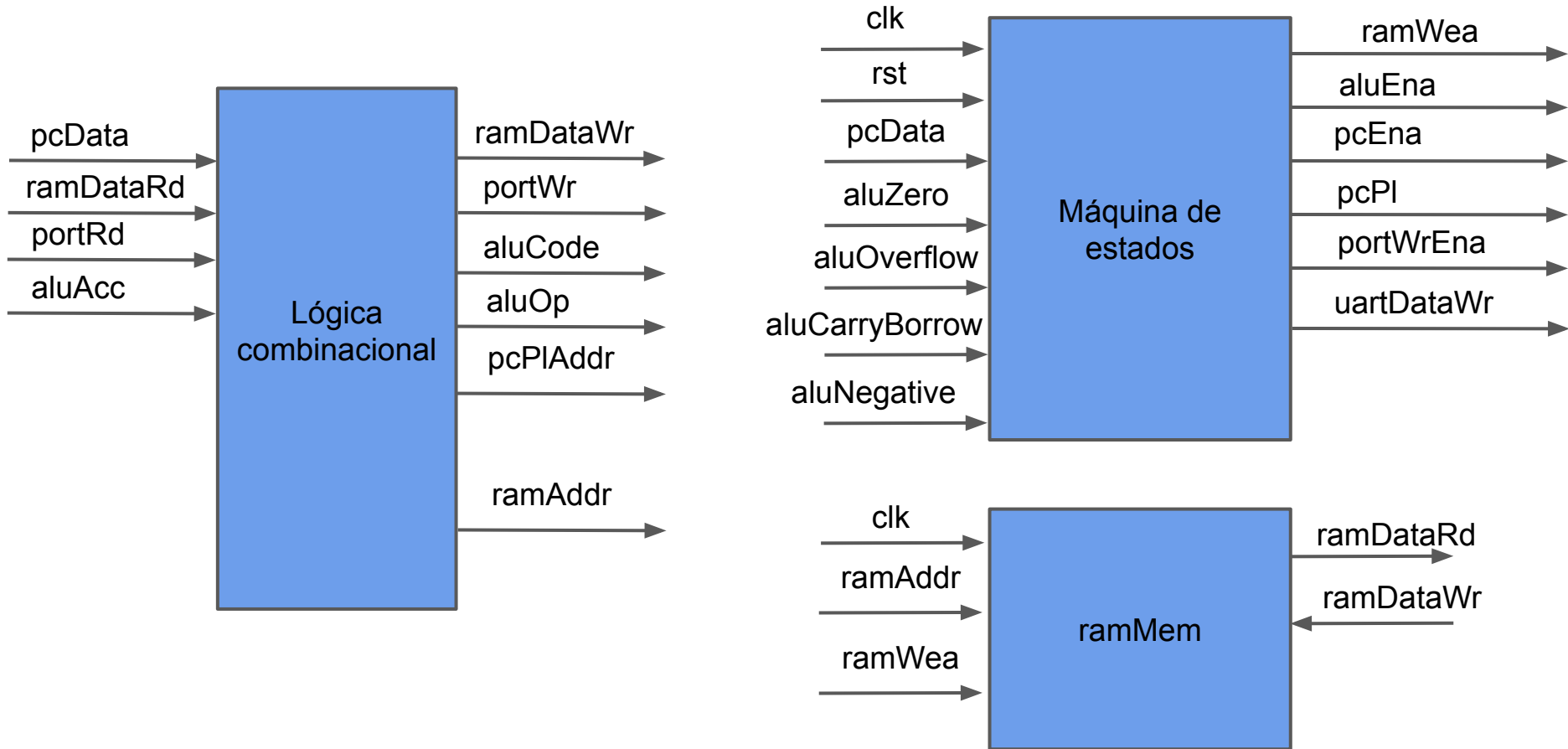


Diagrama en bloques lógica combinacional

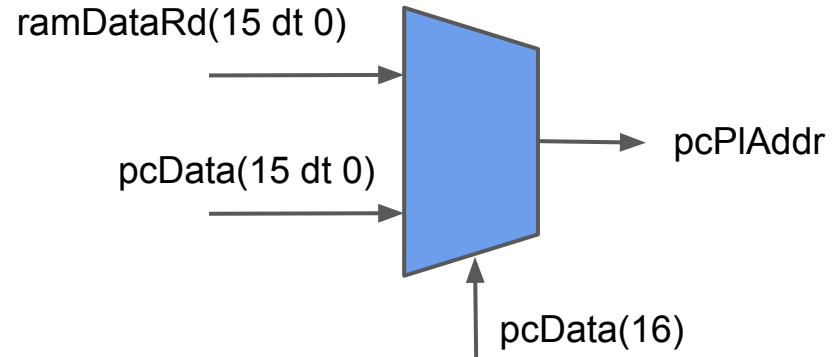
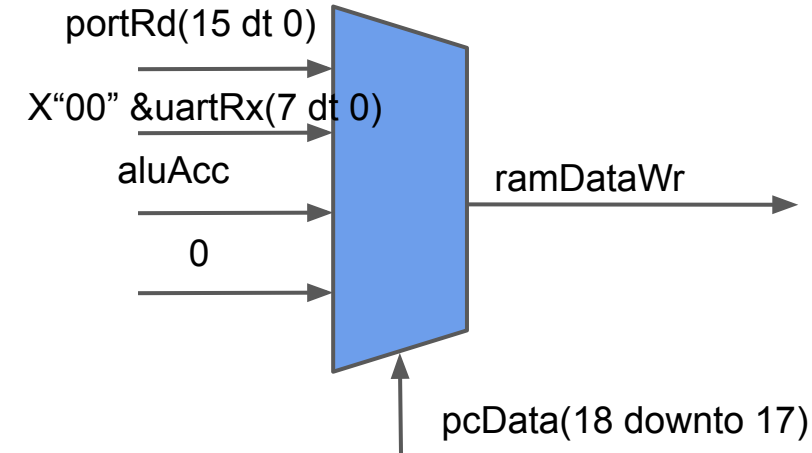
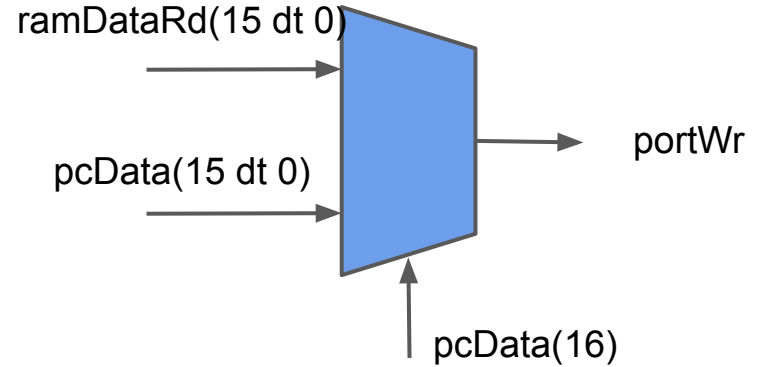
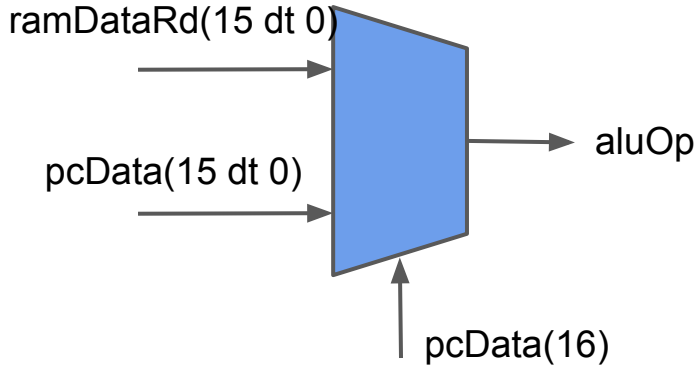
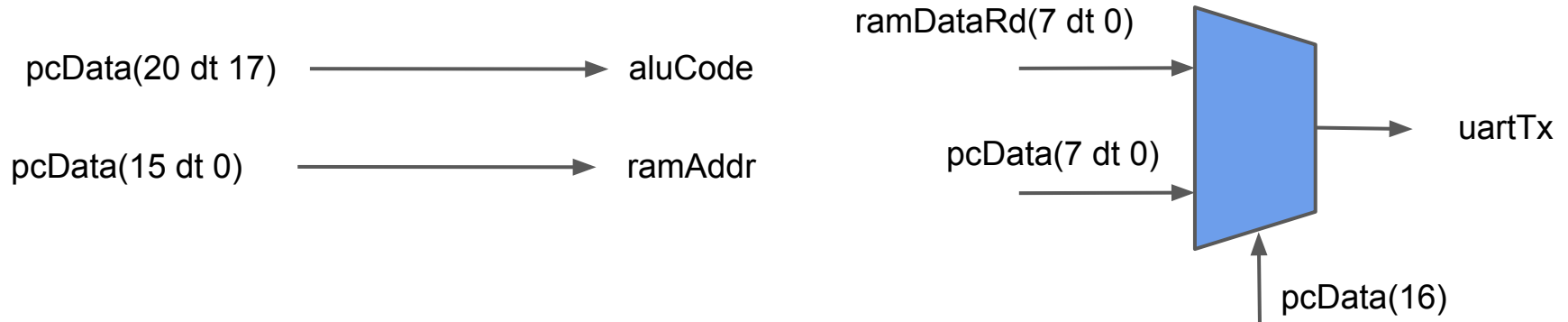


Diagrama en bloques lógica combinacional



Configuración de la block ram para la memoria de RAM

Component Name

Basic

Port A Options

Other Options

Summary

Interface Type

☐ Generate address interface with 32 bits

Memory Type

☐ Common Clock

ECC Options

ECC Type

☐ Error Injection Pins

Write Enable

☐ Byte Write Enable

Byte Size (bits)

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives.
Refer datasheet for more information.

Algorithm

Primitive

Configuración de la block ram para la memoria de RAM

Component Name ramMem

Basic **Port A Options** **Other Options** **Summary**

Memory Size

Write Width 16 Range: 1 to 4608 (bits)

Read Width 16

Write Depth 1024 Range: 2 to 1048576

Read Depth 1024

Operating Mode Write First Enable Port Type Always Enabled

Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

Port A Output Reset Options

☐ RSTA Pin (set/reset pin) Output Reset Value (Hex) 0

☐ Reset Memory Latch Reset Priority CE (Latch or Register Enable)

READ Address Change A

☐ Read Address Change A

Configuración de la block ram para la memoria de RAM

Component Name

Basic **Port A Options** **Other Options** **Summary**

Pipeline Stages within Mux Other Options size: 1x1

Memory Initialization

☐ Load Init File

Coe File

☒ Fill Remaining Memory Locations

Remaining Memory Locations (Hex)

Structural/UniSim Simulation Model Options

Defines the type of warnings and outputs are generated when a read-write or write-write collision occurs.

Collision Warnings

Behavioral Simulation Model Options

☐ Disable Collision Warnings ☐ Disable Out of Range Warnings

Condiciones de entrega

Condiciones de entrega

1. Diseño completo funcionando todas las instrucciones.
2. Testbench de las siguientes unidades
 - a. pc
 - b. portIO
 - c. alu
 - d. uart
 - e. microIO
3. Diagrama de estados de la unidad uc
4. Todo el diseño debe estar subido al repositorio dentro de una carpeta con el nombre microIO
5. Fechas de entrega. Debe estar subido al repositorio antes de estas fechas.
 - a. Primera entrega: 27/11/2025
 - b. Segunda entrega: 18/12/2025
 - c. Última entrega: 10/02/2026
6. Luego de la entrega se coordina un coloquio el que consistirá de las siguientes etapas
 - a. Explicación de lo realizado
 - b. Demostración de funcionamiento con test bench.
 - c. Verificación de timing del diseño.
 - i. Frecuencia máxima de clock slack > 10% periodo del clock del sistema.
 - ii. Velocidad de ejecución
 - d. Preguntas y realización de modificaciones al diseño.

Preguntas??

FIN